Test Plan — Program 3: Refactoring Logic to Read Binary Data

1. Objective

This program counts the number of right triangles that can be formed from a set of 2D points. A right triangle is counted when the right angle occurs at a "pivot" point. Program 3 supports both text input and binary input (.dat) through a PointStore interface, and supports parallel execution through threads and processes by dividing the pivot index range among workers. Produces the same triangle count as Program 0 and Program 1 for the same input file.

2. Major Refactor / Optimization O(n^3) -> O(n^2)

My original implementation used the dot-product method to check every triple of points (i, j, k) to determine whether they formed a right triangle. This required three nested loops, resulting in $O(n^3)$ time complexity. While this approach is correct, it does not scale. As the number of points n increases, the runtime grows cubically:

- If n doubles, runtime increases by approximately **8x**
- If n triples, runtime increases by approximately **27x**

When testing with larger datasets, I observed that the dot-product approach "explodes" quickly as n increases. Even moderate increases in input size caused dramatic increases in execution time. This makes the $O(n^3)$ solution impractical for large inputs.

To solve this, I refactored the algorithm to an **$O(n^2)$** expected-time solution by changing the strategy. Instead of checking all triples directly, I would:

- Fix one pivot point i
- Compute direction vectors from pivot i to every other point.
- Reduce each vector (dx, dy) to its normalized form using gcd(|dx|, |dy|).
- Store counts of each direction in a hash map.
- For each direction (dx, dy), check for its perpendicular direction (dy, -dx) and multiply their frequencies to count right-angle pairs.

This eliminates the innermost loop and reduces the complexity from cubic to quadratic. With the new approach: If n doubles, runtime increases by approximately 4x. If n triples, runtime increases by approximately 9x.

This improvement makes the program scalable to much larger datasets and significantly improves performance for both the single-threaded and parallel implementations.

Additionally, direction vectors are packed into a single long using bit manipulation instead of using String keys. This reduces object creation overhead and further improves performance inside the critical $O(n^2)$ loop.

3. File Input System (PointStore)

The program uses PointStore to abstract point access. The program chooses the correct implementation based on file type:

TextPointStore: Used for all non-.dat files (plain text). Expected format:

- First line: number of points n
- Next n lines: x y integer pairs
- This store reads the file once and provides indexed access through getX(i) and getY(i).

BinPointStore: Used only for .dat files (binary). Expected binary format:

- first 4 bytes = n, followed by n pairs of 4-byte integers (x, y)
- raw (x, y) int pairs until EOF
- The store validates the binary file. If the file is corrupted, too short, or malformed, it throws an IOException and the program prints: Error: Invalid file format

4. Automated Test Execution Results

All functionality was verified using an automated Bash test script (run_tests.sh) that executed all three programs (Triangles, ThreadTriangles, and ProcessTriangles) against a comprehensive test suite. The script validates correctness, performance, parallel behavior, binary/text consistency, and error handling.

A total of **141 individual tests** were executed.

Overall Results

- **Total Tests:** 141
- **Passed:** 141
- **Failed:** 0

All tests completed successfully.

4.1 Correctness Tests (Small and Structured Inputs)

Multiple structured datasets were used to verify correctness, including:

- Spec list (test_spec_list-1.txt/.dat)
- Two-point input
- Collinear points
- Square configurations
- Giant triangle dataset

All expected triangle counts matched exactly across:

- Triangles
- ThreadTriangles (1, 2, 4, 8 threads)
- ProcessTriangles (1, 2, 4, 8 processes)

Both text and binary formats produced identical results.

Example:

- Square binary input (square.dat) correctly returned **4**
- Giant triangle input returned **12**
- Two-point and non-triangle inputs correctly returned **0**

4.2 Large Dataset Performance

Large datasets such as test_long_list and test_time_list were used to validate both performance and scalability.

Results confirmed:

- $O(n^2)$ refactored algorithm completes within required time limits
- Parallel versions demonstrate speedups with increased threads/processes
- Binary input performs comparably to text input

Representative timing:

- test_long_list:
    - Single-thread: ~4–5 seconds
    - Multithreaded / multiprocess: ~2–3 seconds

All runs were well below the provided limits (120s / 30s).

4.3 Binary (.dat) Validation

Binary datasets were tested extensively, including:

- triangle.dat → 1 triangle
- collinear.dat → 0 triangles
- square.dat → 4 triangles
- scaletest.dat → 886 triangles

Results exactly matched their text equivalents.

A corrupted binary file (bad.dat) was also tested to verify graceful failure:

Error: Invalid file format

All three programs correctly detected the corruption and exited with error code 1.

4.4 Error Handling Tests

The following error scenarios were validated:

Missing File
Error: No such file or directory
No Read Permission
Error: Permission denied
Truncated Input
Error: Fewer coordinates than specified

Each error condition was tested across:

- Triangles
- ThreadTriangles
- ProcessTriangles

All programs reported deterministic error messages and exited gracefully.

4.5 Text vs Binary Consistency

For all datasets, including scaletest, results from .txt and .dat inputs matched exactly across:

- Single-threaded execution
- Multi-threaded execution
- Multi-process execution

This confirms correct behavior of both TextPointStore and BinPointStore.

5. Final Summary

Binary Input Performance Observation (.dat vs .txt)

During testing, it was observed that binary .dat files consistently performed slightly faster than their .txt counterparts, especially on larger datasets. This improvement occurs because binary input avoids expensive string parsing and conversion operations.

When reading text files, each coordinate must be:

1. Read as characters
2. Tokenized into strings
3. Converted from decimal text into integers

In contrast, .dat files store coordinates directly as 32-bit integers. This allows BinPointStore to read raw bytes and immediately interpret them as numeric values, eliminating parsing overhead.

As a result:

- File I/O is faster
- CPU usage during input processing is reduced
- Large datasets load more efficiently

This performance advantage becomes more noticeable as input size increases and complements the $O(n^2)$ algorithm refactor by minimizing input overhead for large-scale tests.